

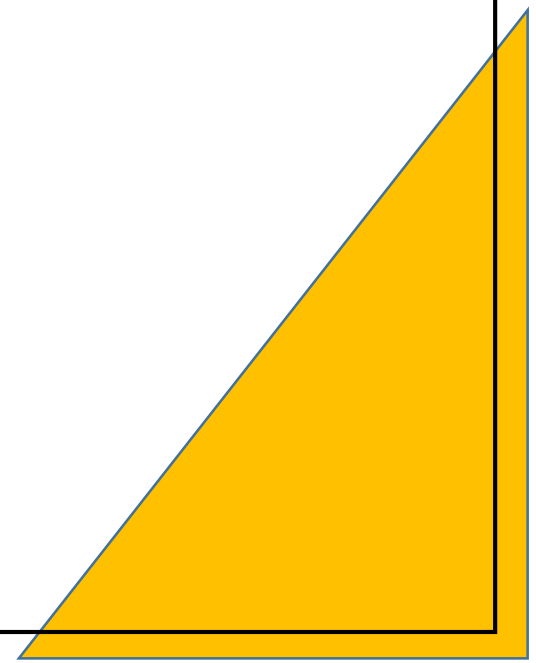
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INTELIGENCIA ARTIFICIAL

Prof. Me. Matheus Raffael Simon

Busca Heurística

Capítulo 3 – Russell & Norvig



Estratégias de Busca Exaustiva (Cega)

- Encontram soluções para problemas pela geração *sistemática* de novos estados, que são comparados ao objetivo;
- São *ineficientes* na maioria dos casos:
  - utilizam apenas o *custo de caminho* do nó atual ao nó inicial (função *g*) para decidir qual o próximo nó da fronteira a ser expandido.
  - essa medida nem sempre conduz a busca na direção do objetivo.
- Como encontrar um barco perdido?
  - não podemos procurar no oceano inteiro...
  - observamos as correntes marítimas, o vento, etc...

Estratégias de busca heurística (Informada)

- Utilizam conhecimento específico do problema na escolha do próximo nó a ser expandido
  - barco perdido: correntes maritimas, vento, etc
- Aplicação de uma função de avaliação a cada nó na fronteira do espaço de estados
  - essa função ESTIMA o custo de caminho do nó atual até o objetivo mais próximo utilizando uma FUNÇÃO HEURÍSTICA.

Busca Heurística

- Classes de algoritmos para busca heurística:
  1. Busca pela melhor escolha (*Best-First search*)
  2. Busca com limite de memória
  3. Busca com melhora iterativa

Busca pela Melhor Escolha

Best-First Search

- Busca pela Melhor Escolha - BME
  - Busca genérica onde o nó de menor custo "aparente" na fronteira do espaço de estados é expandido primeiro
- Duas abordagens básicas:
  - 1. Busca Gulosa (Greedy search)
  - 2. Algoritmo A*

Busca pela Melhor Escolha

Algoritmo geral

- Função-Insere
  - insere novos nós na fronteira ordenados com base na Função-Avaliação
    - Que está baseada na função heurística

função Busca-Melhor-Escolha **(***problema*, *Função-Avaliação***)**

Busca-Genérica (*problema, Função-Insere*)

retorna **uma solução**

BME: Busca Gulosa

- Semelhante à busca em profundidade com *backtracking*
- Tenta expandir o nó mais próximo do nó final com base na estimativa feita pela função heurística *h*
- *Função-Avaliação*
  - função heurística *h*

Funções Heurísticas

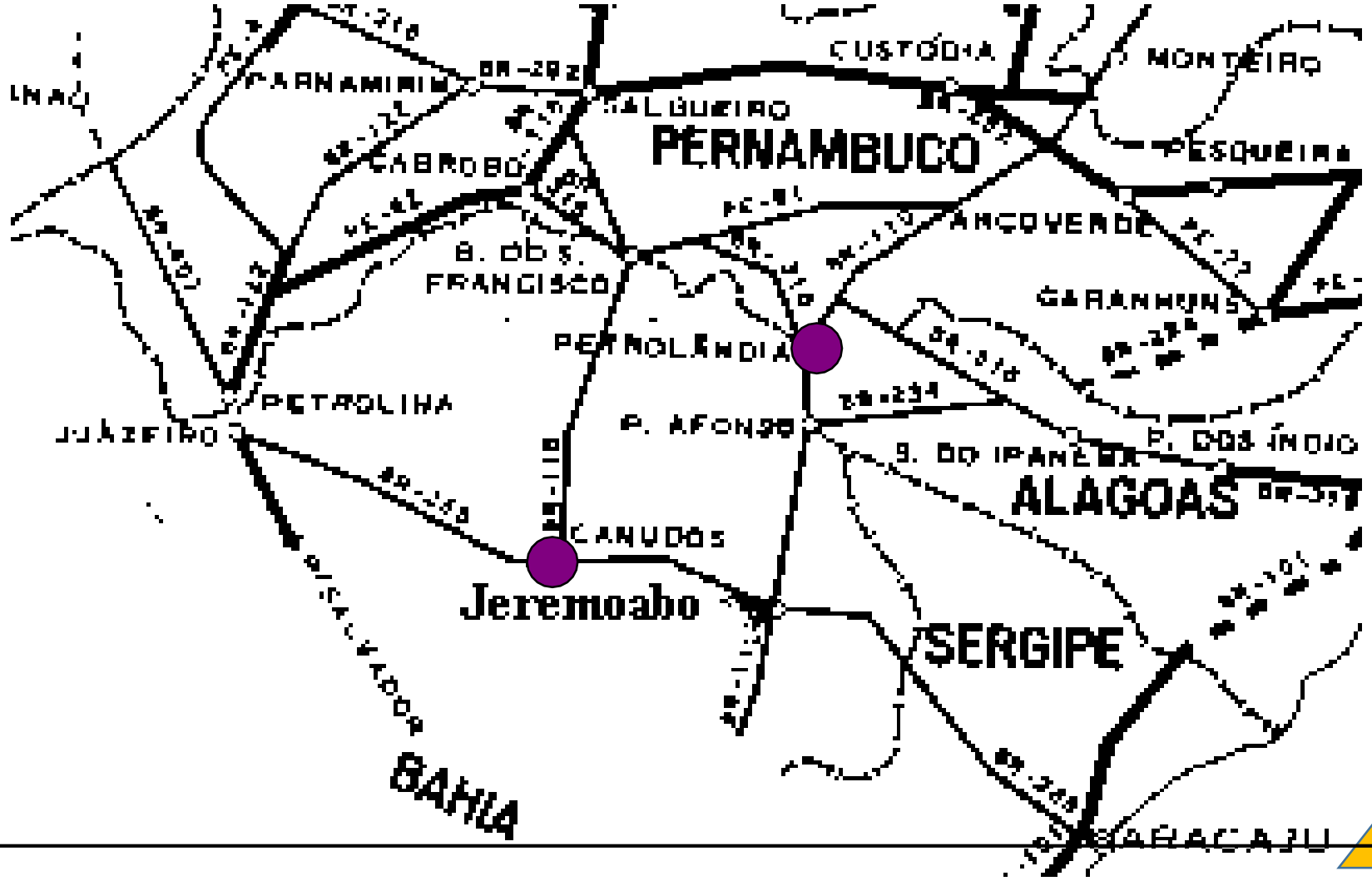
- Função heurística - *h*
  - estima o custo do caminho mais barato do estado atual até o estado final mais próximo.
- Funções heurísticas são específicas para cada problema
- Exemplo:
  - encontrar a rota mais barata de Canudos a Petrolândia
  - $h_{dd}(n)$ = distância direta entre o nó *n* e o nó final.

Funções Heurísticas

- Como escolher uma boa função heurística?
  - ela deve ser admissível
  - i.e., nunca *superestimar* o custo real da solução
- Distância direta (h_{dd}) é *admissível* porque o caminho mais curto entre dois pontos é sempre uma linha reta

Exemplo: encontrar a rota mais barata de Canudos a Petrolândia

hdd(n) = distância direta entre o nó n e o nó final



Busca Gulosa

- Custo de busca mínimo!
  - não expande nós fora do caminho
- Porém *não* é ótima:
  - escolhe o caminho que é mais econômico à primeira vista
    - Belém do S. Francisco, Petrolândia = 4,4 unidades
  - porém, existe um caminho mais curto de Canudos a Petrolândia
    - Jeremoabo, P. Afonso, Petrolândia = 4 unidades
- A solução via Belém do S. Francisco foi escolhida por este algoritmo porque
  - $h_{dd}(BSF) = 1,5$ *u.,* enquanto $h_{dd}(Jer) = 2,1$ u.

Busca Gulosa

- Não é completa:
  - pode entrar em *looping* se não detectar a expansão de estados repetidos
  - pode tentar desenvolver um caminho infinito

- Custo de tempo e memória: $O(b^d)$
  - guarda todos os nós expandidos na memória

BME: Algoritmo A*

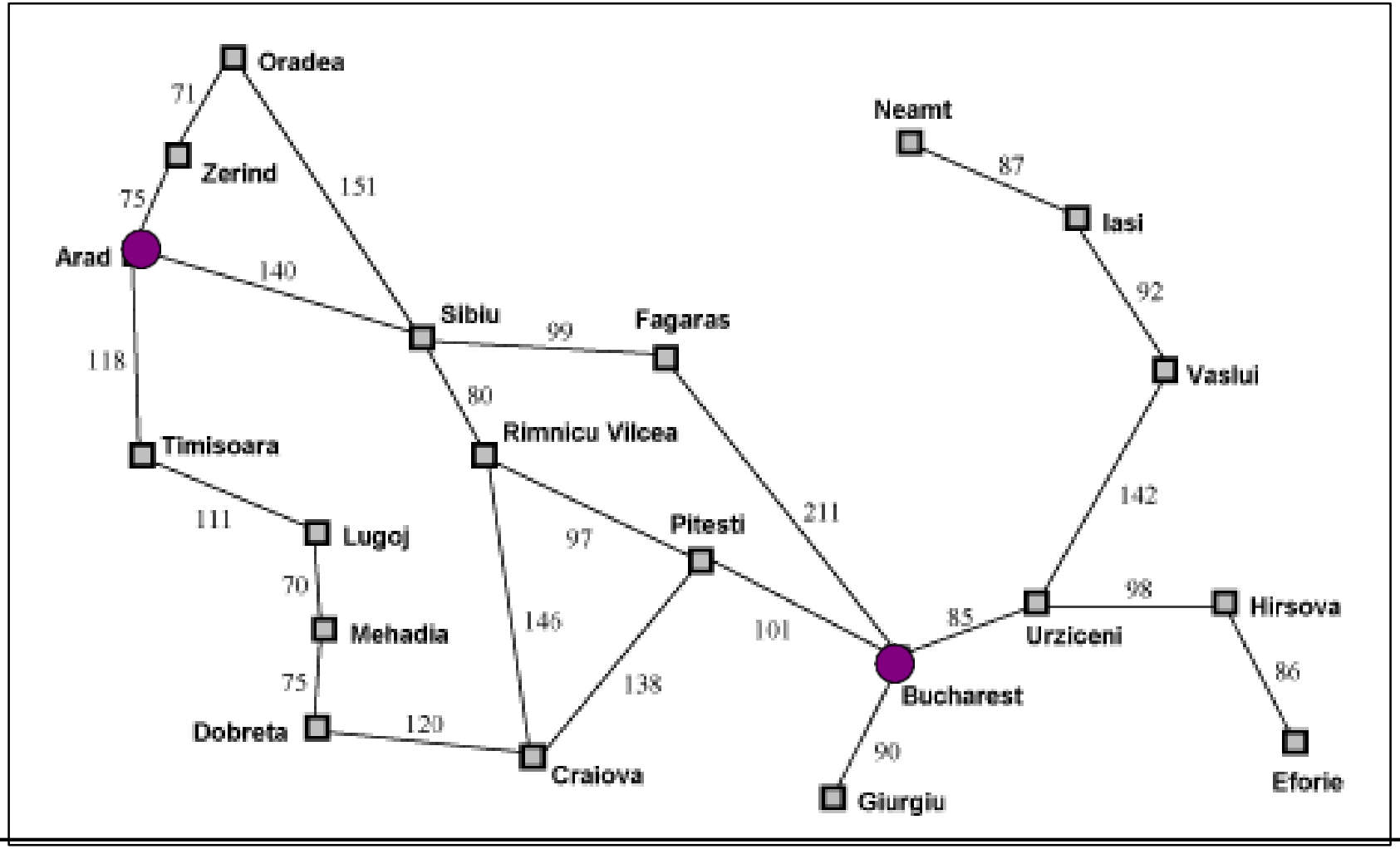
- A* expande o nó de menor valor de *f* na fronteira do espaço de estados
- Tenta minimizar o custo total da solução combinando:
  - Busca Gulosa (h)
    - econômica, porém não é completa nem ótima
  - Busca de Custo Uniforme (g)
    - ineficiente, porém completa e ótima
- *f* - Função de avaliação do A*:
  - *f (n) = g (n) + h (n)*
  - *g (n)* = distância de *n* ao nó inicial
  - *h (n)* = distância estimada de *n* ao nó final

Algoritmo A*

- Se *h* é *admissível*, então *f (n)* é admissível também
  - i.e., *f* nunca irá superestimar o custo real da melhor solução através de *n*
  - pois *g* guarda o valor exato do caminho já percorrido.
- Com A*, a rota escolhida entre *Canudos* e *Petrolândia* é de fato a mais curta, uma vez que:
  - *f (BSF) = 2,5 u + 1,5 u = 4 u*
  - *f (Jeremoabo) = 1,5 u + 2,1 u = 3,6 u*

Algoritmo A*: outro exemplo

Viajar de Arad a Bucharest



Straight line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca A*: exemplo

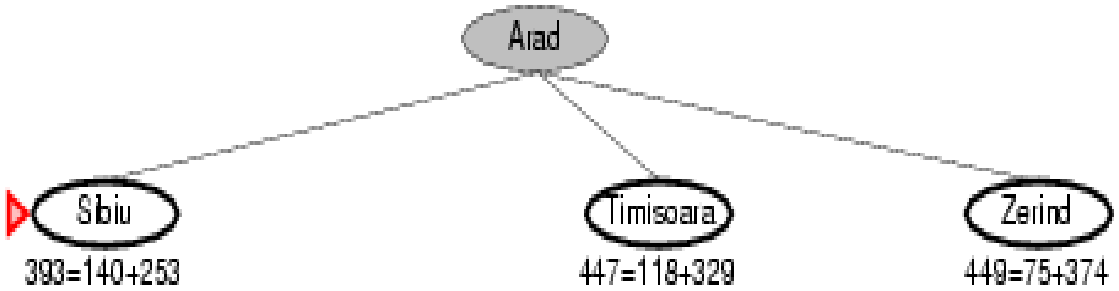


A busca A* combina duas funções de custo:
f(n)=g(n)+h(n)

- g(n) custo real do caminho do início até o nó n.
- h(n) heurística, uma estimativa de custo de n até o objetivo.f(n)
- f(n) custo total estimado passando por n.

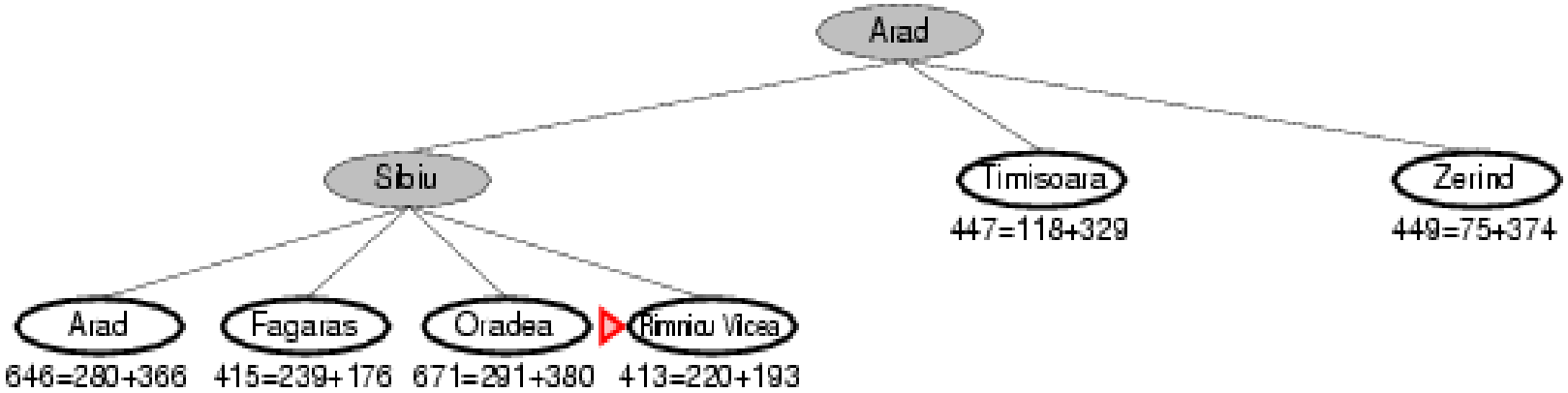
Ponto de partida: Arad.
Cálculo:
g(Arad) = 0 (custo real do início até aqui).
h(Arad) = 366 (estimativa até Bucareste).
f(Arad) = g + h = 0 + 366 = 366.
Nós na fronteira: apenas Arad (f=366).

Busca A*: exemplo



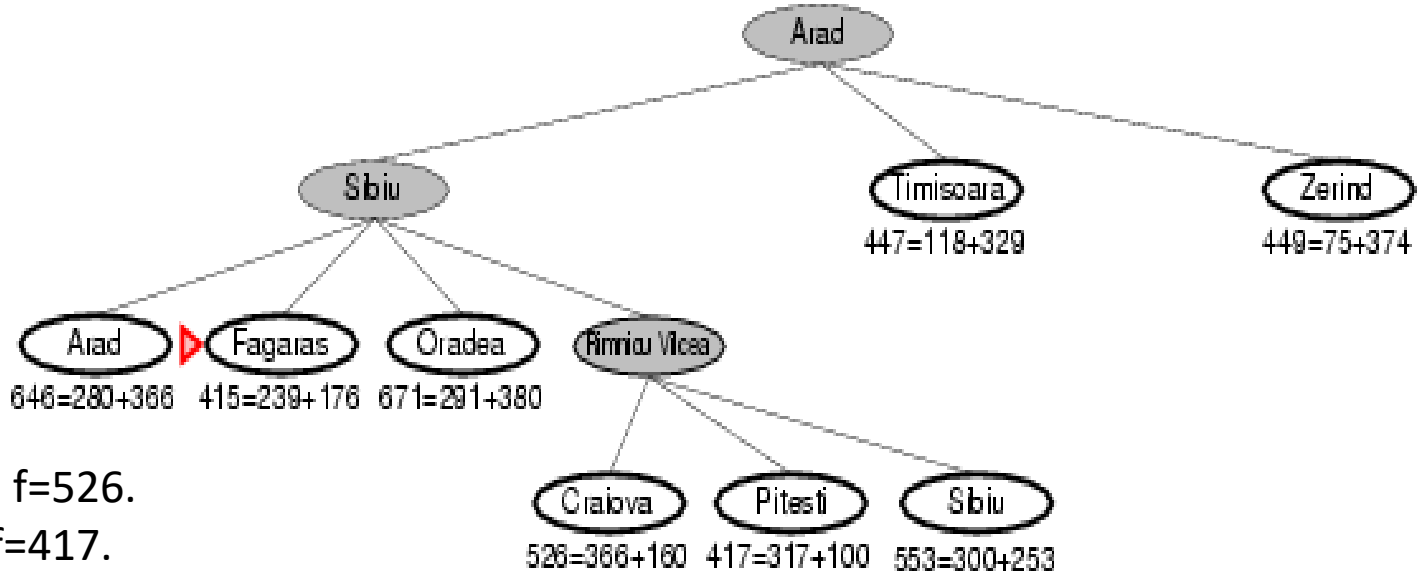
Sibiu: g=140, h=253 → f=393.
Timișoara: g=118, h=329 → f=447.
Zerind: g=75, h=374 → f=449.
Menor f: Sibiu (393) → próximo a expandir.

Busca A*: exemplo



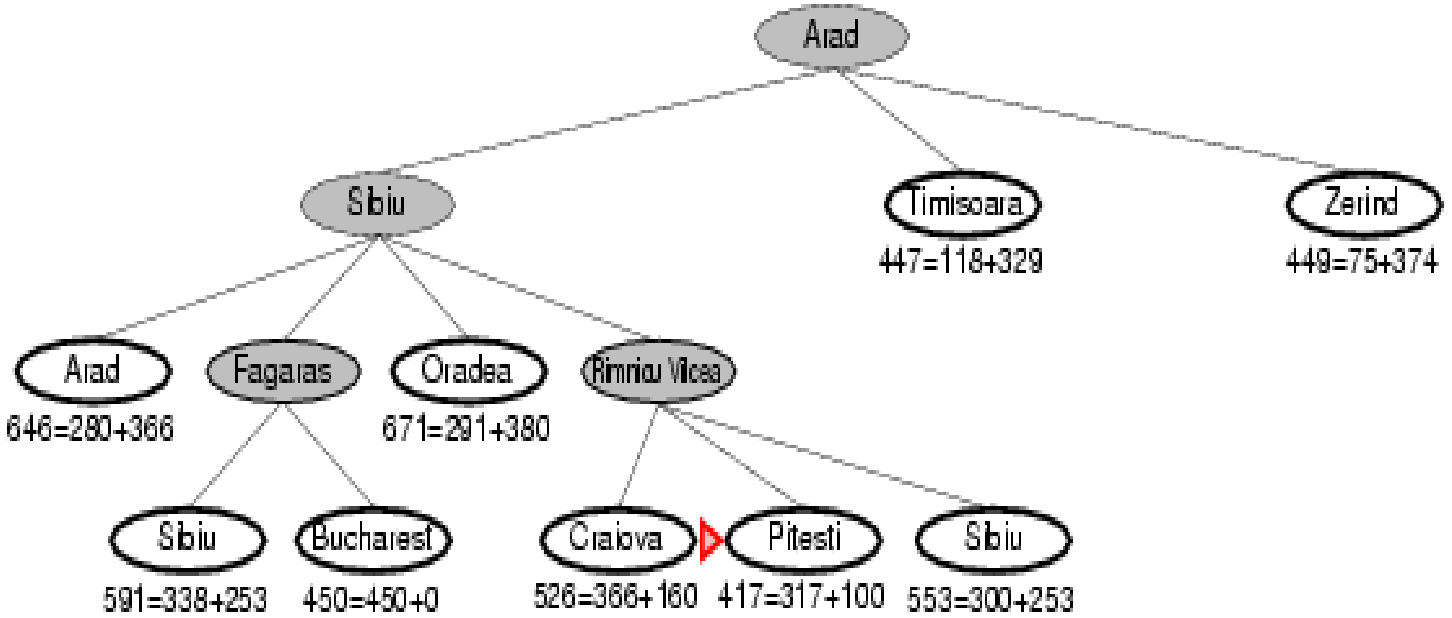
Arad: g=280, h=366 → f=646.
Făgăraș: g=239, h=176 → f=415.
Oradea: g=291, h=380 → f=671.
Rimnicu Vîlcea: g=220, h=193 → f=413.
Menor f: **Rimnicu Vîlcea (413)** → próximo a expandir.

Busca A*: exemplo



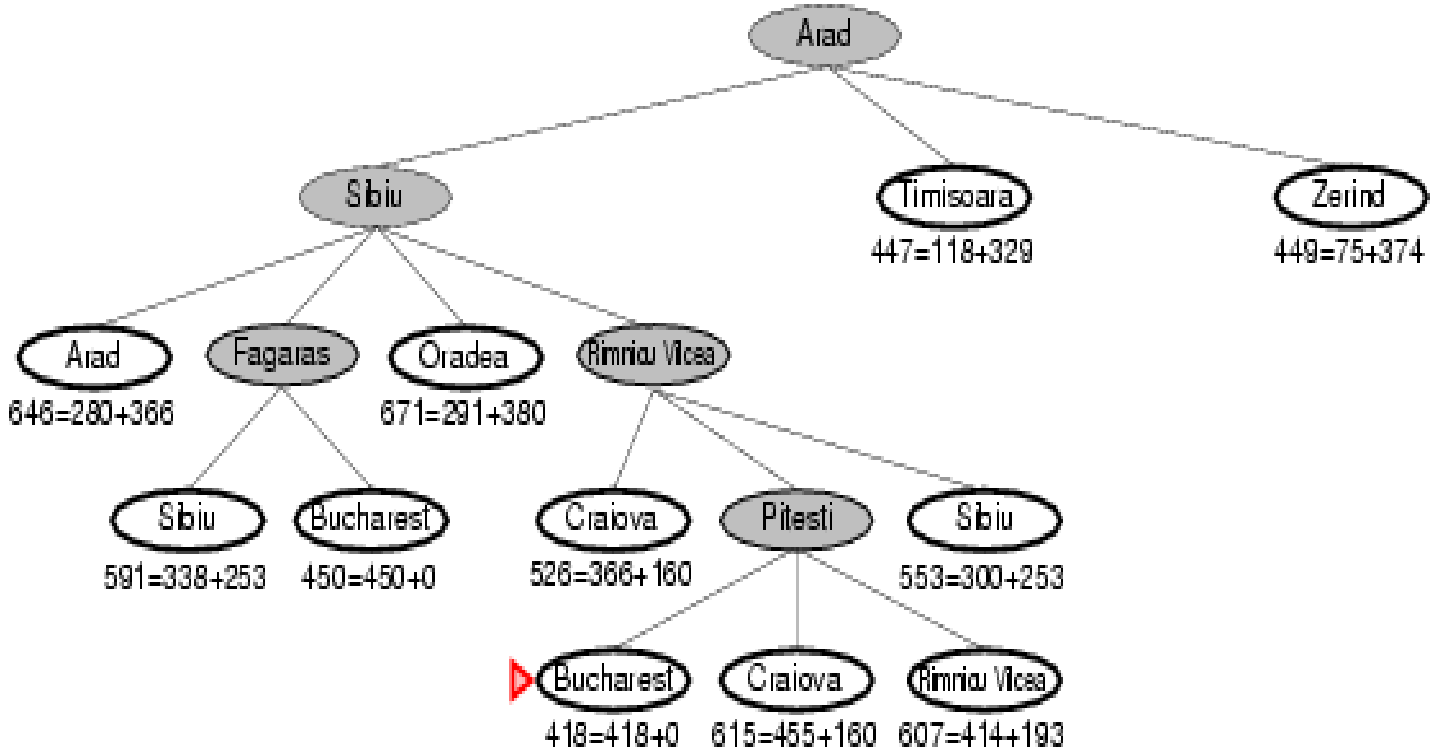
Craiova: g=366, h=160 → f=526.
Pitești: g=317, h=100 → f=417.
Sibiu: g=300, h=253 → f=553.
Menor f: **Pitești (417)** → próximo a expandir.

Busca A*: exemplo

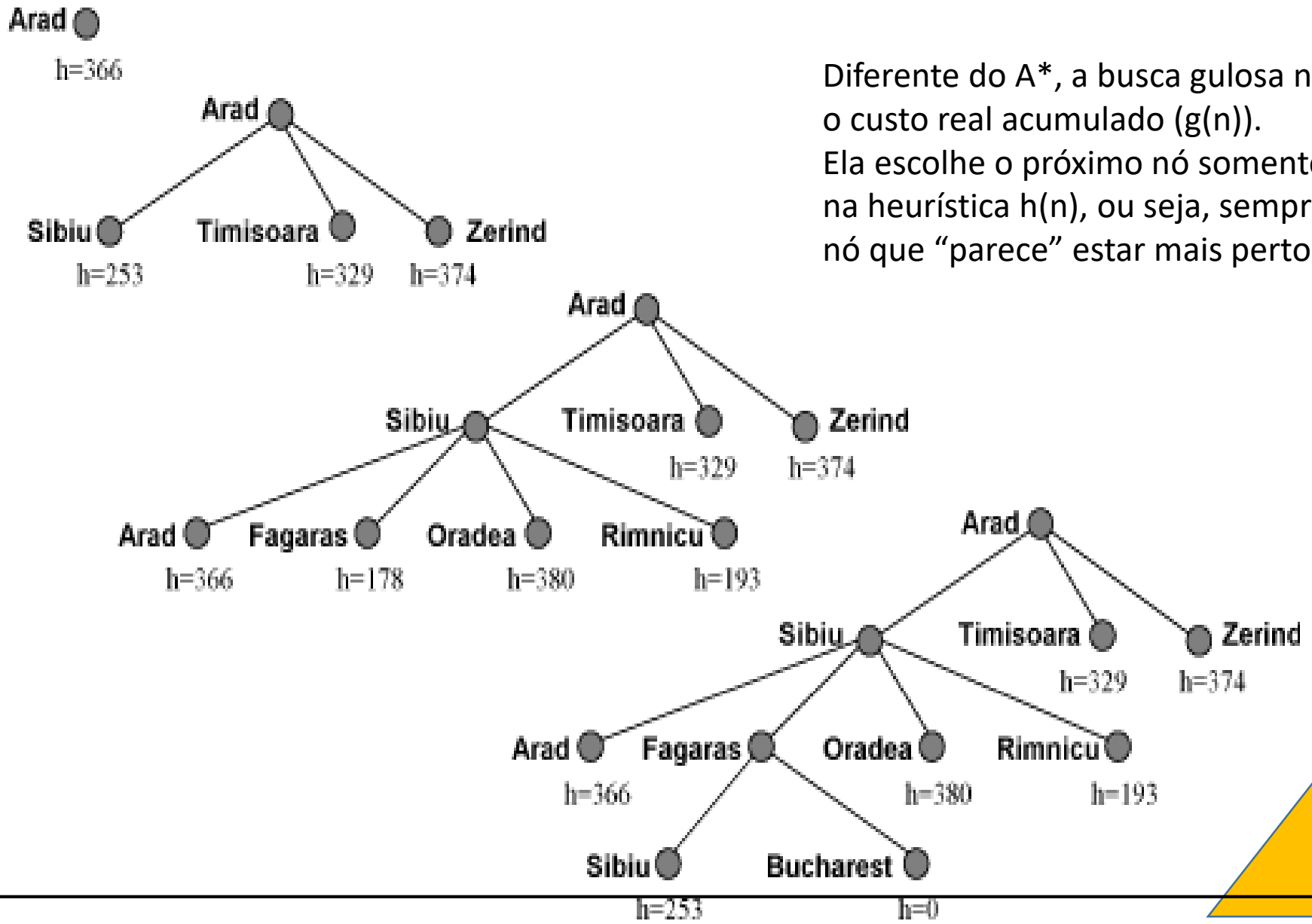


Bucareste: g=418, h=0 → f=418.
Craiova: g=455, h=160 → f=615.
Rimnicu Vîlcea: g=414, h=193 → f=607.
Encontrado o objetivo: **Bucareste (418)**.

Busca A*: exemplo

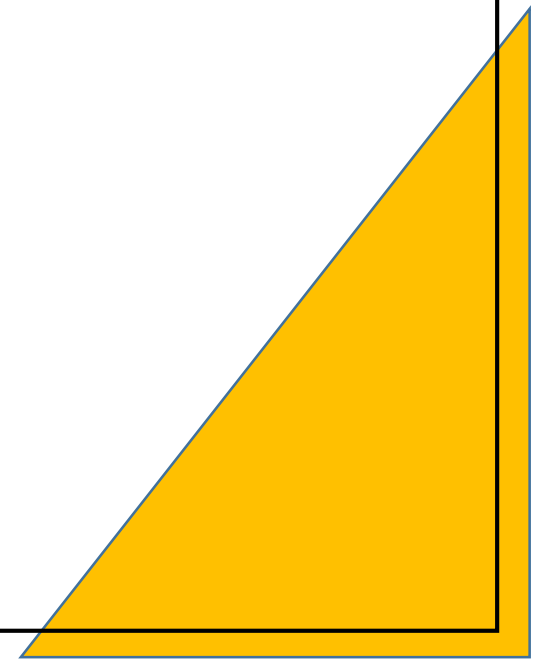


Se fosse pela Busca Gulosa...



Diferente do A*, a busca gulosa não considera o custo real acumulado (g(n)).
Ela escolhe o próximo nó somente com base na heurística h(n), ou seja, sempre expande o nó que “parece” estar mais perto do objetivo.

Perceberam que A* tomou um rumo diverso do algoritmo guloso??
A* tomou o caminho ótimo..



Algoritmo A*:
Análise do comportamento

- A estratégia é completa e ótima
- Custo de tempo:
  - exponencial com o comprimento da solução, porém boas funções heurísticas diminuem significativamente esse custo
    - o fator de expansão fica próximo de 1
- Custo de memória*:* $O(b^d)$
  - guarda todos os nós expandidos na memória, para possibilitar o *backtracking*

Algoritmo A*

Análise do comportamento

- A estratégia apresenta eficiência ótima
  - nenhum outro algoritmo ótimo garante expandir menos nós
- A* só expande nós com $f(n) \leq C^*$, onde C^* é o custo do caminho ótimo
- Para se garantir otimalidade do A*, o valor de f em um caminho particular deve ser não decrescente!!!
  - $f(\text{sucessor}(n)) \geq f(n)$
  - i.e., o custo de cada nó gerado no mesmo caminho nunca é menor do que o custo de seus antecessores

Algoritmo A*

Análise do comportamento

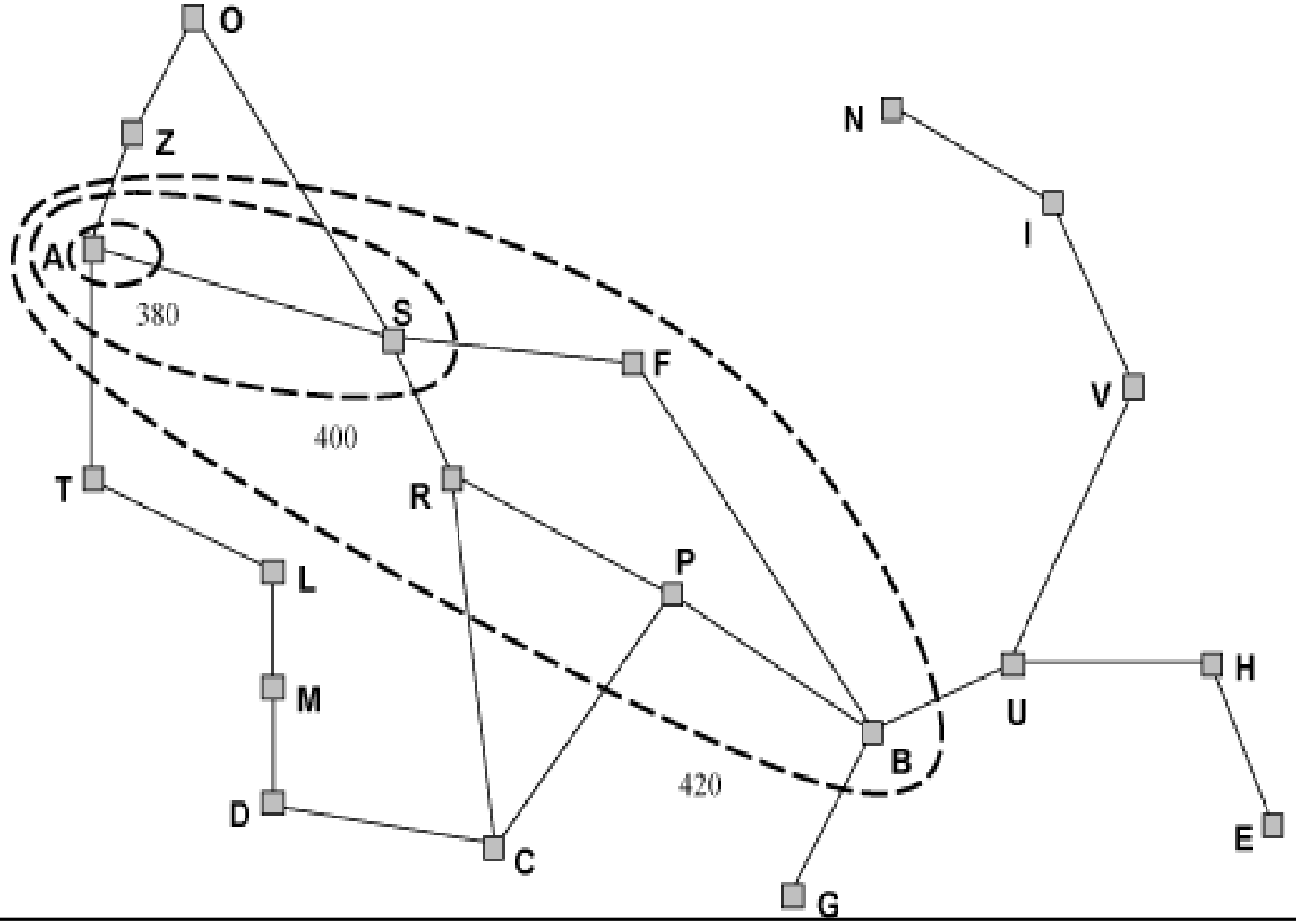
- $f = g + h$ deve ser não decrescente
  - g é não decrescente (para operadores não negativos)
    - custo real do caminho já percorrido
  - h deve ser não-crescente (consistente, monotônica)
    - h (n) $\geq$ h (sucessor(n))
    - i.e., quanto mais próximo do nó final, menor o valor de h
    - isso vale para a maioria das funções heurísticas
- Quando h não é consistente, para se garantir otimalidade do A*, temos:
  - quando f(suc(n)) < f (n)
  - usa-se f(suc(n)) = max (f(n), g(suc(n)) + h(suc(n)))

A* define Contornos

$f(n) \leq C^*$

fator de expansão

próximo de 1



A* expande os nós em camadas de custo f(n) = g(n) + h(n). As elipses representam os limites de custo (380, 400 e 420). Assim, o algoritmo considera primeiro os nós mais promissores, próximos de Arad, e vai ampliando a fronteira até alcançar Bucareste. Isso ilustra que o A* é eficiente, pois foca na região mais promissora e garante encontrar o caminho ótimo.

Trabalho

- *Deve-se dividir a sala em 3 grupos.*
- *Cada equipe deve desenvolver e apresentar os algoritmos abaixo*
- *Busca Gulosa.*
- *Algoritmo A*.*
- *Busca Cega (profundidade).*

- *Cada grupo deverá apresentar, 15 minutos no máximo, o algoritmo, sua utilidade e softwares que utilizam tal solução. Além de criar um exemplo/problema e executar ele durante a apresentação.*
- *Pode ser em qualquer linguagem.*
- *Deverão ainda explicar as partes principais do algoritmo.*
- *Todos deverão explicar uma parte, na apresentação e no código.*
- *Apresentação será na ultima aula antes da prova.*

Procurem artigos e não o GPT ☺